PROPUESTA TÉCNICA · PLATAFORMA DE AGENTES DE IA

Agentes como configuración gobernada

Una capa común para crear, operar y controlar los agentes de IA de una comercializadora de energía

Daniel Felipe Diab Gonzalez

Octubre de 2026

TL;DR · Resumen ejecutivo

- **El problema.** La empresa ya usa IA en facturación, cobranzas y ventas, pero cada área la armó por su cuenta. Nadie sabe cuántos agentes hay, con qué modelo corren, qué datos de clientes tocan ni cuánto cuestan.
- **La propuesta.** Una plataforma común donde cada agente es un *manifiesto*: una ficha versionada que dice qué hace, de quién es, qué herramientas y datos usa y cuánto puede gastar. Sin código y sin prompts escritos a mano.
- **El control.** Un solo camino a producción con tres puntos obligatorios: registro, evaluación y gateway. El camino fácil es el camino controlado.
- **La experiencia.** Cualquier área arma su agente conversando con un asistente, lo prueba y lo publica; sus usuarios reciben respuestas fluidas, con datos reales.
- **Simple a propósito.** Un runtime (LangGraph), un gateway, una base de datos y una API para todos los agentes: menos costo, menos piezas que fallen, menos legado.
- **El plan.** Visibilidad total en 2 semanas, primer agente certificado (facturación) en 2 meses y autoservicio para todas las áreas en 4. Desde ahí, cambiar de modelo deja de ser un riesgo: cada cambio pasa por evaluación y se revierte en un paso.

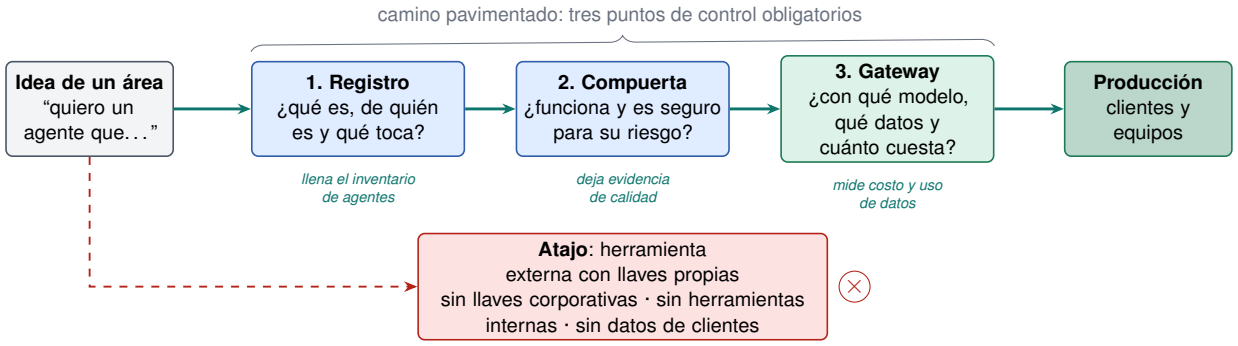


Figura 1: La propuesta en un diagrama: el camino más fácil a producción es también el único que tiene llaves, herramientas internas y datos de clientes.

Índice

1. Situación actual . . . 2
2. Arquitectura propuesta . . . 2
3. Definición de agente . . . 10
4. Creación de agentes sin escribir prompts . . . 11
5. Evaluación y monitoreo . . . 12
6. Caso aplicado: asistente de facturación . . . 13
7. Hoja de ruta . . . 15
8. Próximos pasos . . . 16
A. Manifiesto completo del asistente de facturación . . . 18

1 Situación actual

Una comercializadora de energía ya usa IA en varias áreas: un asistente de facturación en servicio al cliente, mensajes automáticos en cobranzas y resúmenes de llamadas en ventas. Cada uno funciona, pero cada uno se armó en una herramienta distinta, con su propio modelo, sus propias llaves y su propio acceso a datos de clientes (figura 2). El resultado es que nadie tiene la vista de conjunto.

Esto tiene nombre en ingeniería de ML: cada agente aislado es un *consumidor no declarado* de datos [1], algo que usa información sensible sin que la organización lo sepa. Y el costo de no resolverlo es concreto: Gartner proyecta que más del 40 % de los proyectos agénticos se cancelarán antes de 2027 por costos que se salen de control, valor poco claro o controles de riesgo insuficientes [2]. Las tres causas son exactamente lo que una capa común vuelve visible y controlable.

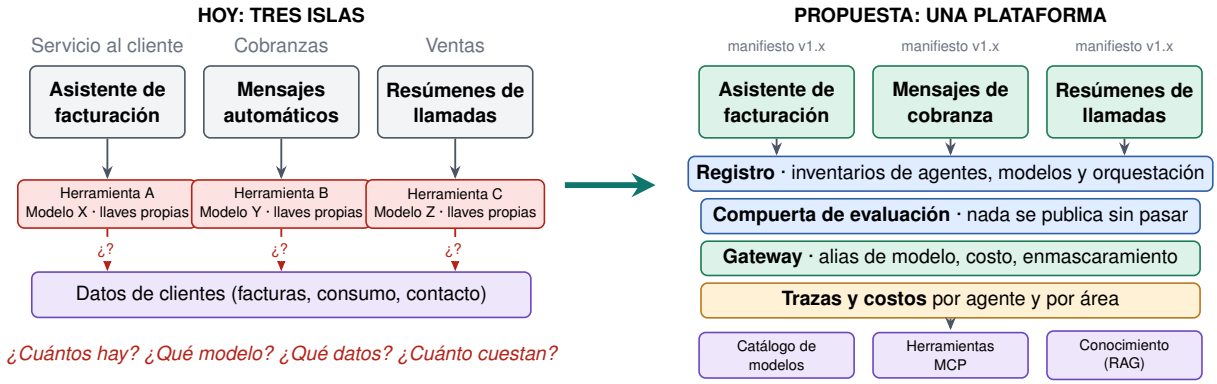


Figura 2: De islas a plataforma: los mismos agentes, declarados en un registro y ejecutados a través de controles comunes.

El documento se organiza en torno a seis preguntas de diseño. Cada sección abre con la respuesta corta y cada decisión importante va acompañada de su razón y de la alternativa que descarté. Varias de esas decisiones vienen de errores que ya vi en sistemas agénticos que construí y operé en producción; cuando es así, lo digo.

2 Arquitectura propuesta

En resumen

Dos planos y una capa transversal. El **plano de control** decide *qué* puede existir (registro, catálogos, constructor, compuerta de evaluación, políticas). El **plano de ejecución** decide *cómo* corre (API de agentes, runtime, gateway de modelos, servidores de herramientas y conocimiento). La **observabilidad** registra todo y devuelve señales al plano de control.

La arquitectura está pensada para que la experiencia sea lo más fluida posible para las dos personas

que la usan: quien crea un agente (lo arma conversando, lo prueba y lo publica sin abrir un ticket) y quien conversa con él (respuestas que fluyen, con datos reales y sin fricción). Parte importante del proceso fue **condensar la arquitectura** para minimizar costos, evitar piezas móviles que puedan fallar y no dejar complejidad de legado a los desarrolladores que vengan después: un solo runtime, un solo gateway, una sola base de datos y una sola API para todos los agentes.

Todas las decisiones que siguen salen de seis principios, y son ellos los que deberían sobrevivir si mañana cambian las herramientas concretas:

P1. El agente es configuración declarativa, no código.
P2. Existe un solo camino a producción, y es el más fácil.
P3. Los agentes piden capacidades, no modelos.
P4. Lo que se recupera (documentos, herramientas) es dato, nunca instrucción.
P5. Nada se publica ni cambia sin evaluación.
P6. La solución más simple que funcione [3].

2.1 Contexto

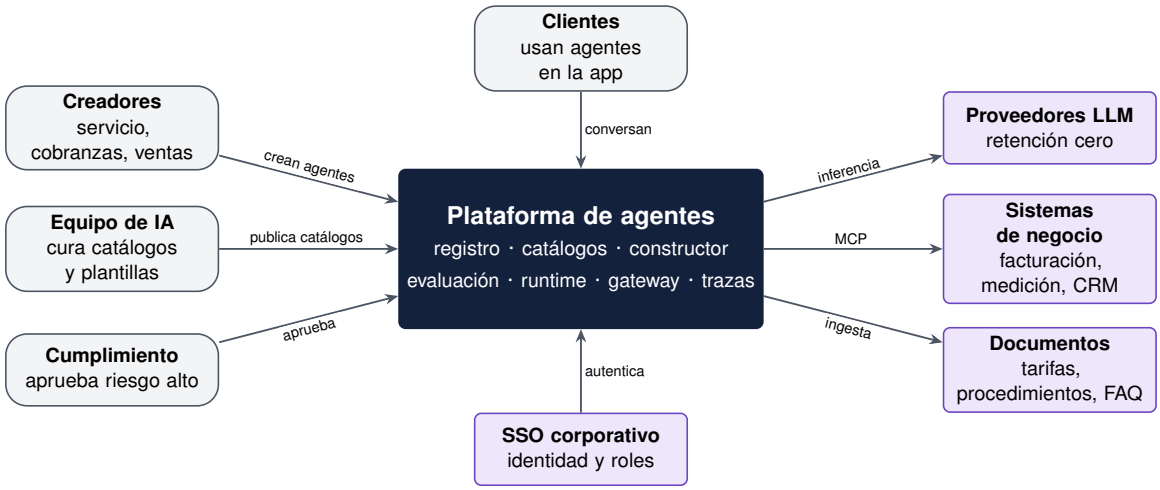


Figura 3: Vista de contexto. A la izquierda, las personas que crean, curan y aprueban; a la derecha, los sistemas externos. La plataforma es el único punto de contacto entre ambos.

Cuatro tipos de personas usan la plataforma con papeles distintos: los creadores de cada área, el equipo de IA que cura catálogos y plantillas, cumplimiento que aprueba lo de riesgo alto, y los clientes que conversan con los agentes publicados (figura 3).

2.2 Componentes y stack

La figura 4 muestra los componentes y la tabla 1 el stack con que los implemento. Estas son las decisiones que los definen.

Decisión: Arquitectura condensada: un runtime, un gateway, una base de datos, una API.
Por qué: Cada pieza adicional es algo más que puede fallar, que hay que pagar y que alguien tendrá que mantener. Con estas cuatro se cubren todos los agentes de la empresa; lo demás se agrega solo cuando un caso real lo pida.
Descarté: Microservicios por capacidad, colas y una base de datos por función desde el primer día.

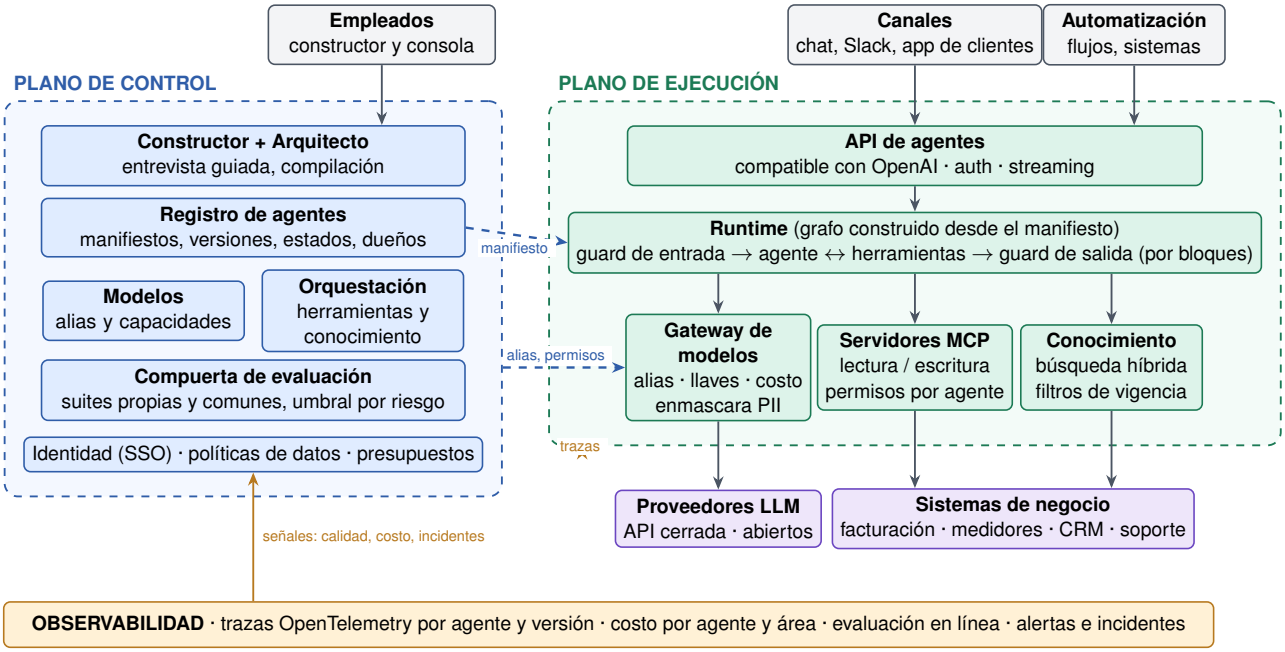


Figura 4: Componentes. Azul: plano de control. Verde: plano de ejecución. Ámbar: observabilidad. Punteado: configuración; continuo: ejecución.

Tabla 1: Stack propuesto. Todo es de código abierto o estándar, y todo corre dentro de la infraestructura de la empresa salvo los proveedores de modelos.

Pieza	Tecnología	Rol
Runtime de agentes	**LangGraph** (Python)	Convierte cada manifiesto en un grafo ejecutable
API y plano de control	FastAPI + Pydantic	API compatible con OpenAI, registro, validación del manifiesto
Gateway de modelos	LiteLLM	Alias, llaves, costo, reintentos y respaldo
Herramientas	Servidores MCP	Conectores a facturación, medición, CRM y soporte
Datos	PostgreSQL + pgvector	Registro, vectores, estado de conversaciones
Observabilidad	Langfuse + OpenTelemetry	Trazas, costos y evaluación en línea
Consola y constructor	Aplicación web	Inventarios, constructor conversacional, aprobaciones

Decisión: LangGraph como framework de agentes.
Por qué: Modela cada agente como un grafo de estados explícito, que es justo lo que produce un manifiesto: se puede leer, probar y auditar paso a paso. Guarda el estado en PostgreSQL, así que una conversación se reanuda después de una aprobación humana (necesario para L4), y trae streaming nativo. Además, ya lo he llevado a producción.
Descarté: Marcos multiagente conversacionales (el flujo queda implícito en la charla entre agentes y es difícil de auditar) y un orquestador propio (otro framework que mantener).

Decisión: Separar plano de control y plano de ejecución.
Por qué: Lo que cambia poco y requiere aprobación (qué agentes existen, con qué permisos) no debe mezclarse con lo que corre miles de veces al día. Se escalan, despliegan y auditan por separado.
Descarté: Un servicio monolítico por agente, que es exactamente la situación actual.

Decisión: El gateway es el único componente con llaves de proveedores.
Por qué: Así cada llamada a un modelo queda atribuida a un agente y un área, con costo y política de datos aplicados en un solo lugar. "¿Cuánto cuesta cada agente?" se vuelve una consulta.
Descarté: Llaves por equipo o por agente: es la causa de que hoy nadie sepa cuánto se gasta.

Decisión: Herramientas expuestas por MCP [4].
Por qué: Cada sistema de la empresa se conecta una vez y queda disponible para todo agente autorizado, con operaciones de lectura y escritura separadas y permisos declarados en el manifiesto.
Descarté: Funciones embebidas en cada agente: se reimplementan, se revisan N veces y nadie sabe quién puede escribir en qué.

Decisión: API de agentes compatible con OpenAI.
Por qué: Cada agente publicado aparece como un "modelo": cualquier canal (chat interno, Slack, app de clientes) se conecta sin integración a la medida. Ya lo he operado en producción.
Descarté: Una API propia por canal.

2.3 Una consulta de punta a punta

Proceso intervenido: consulta de factura en servicio al cliente. Hoy un asesor revisa facturación, medición y la normativa tarifaria para responder. Con la plataforma, el asistente responde con los datos del cliente y escala a un asesor lo que no le corresponde.

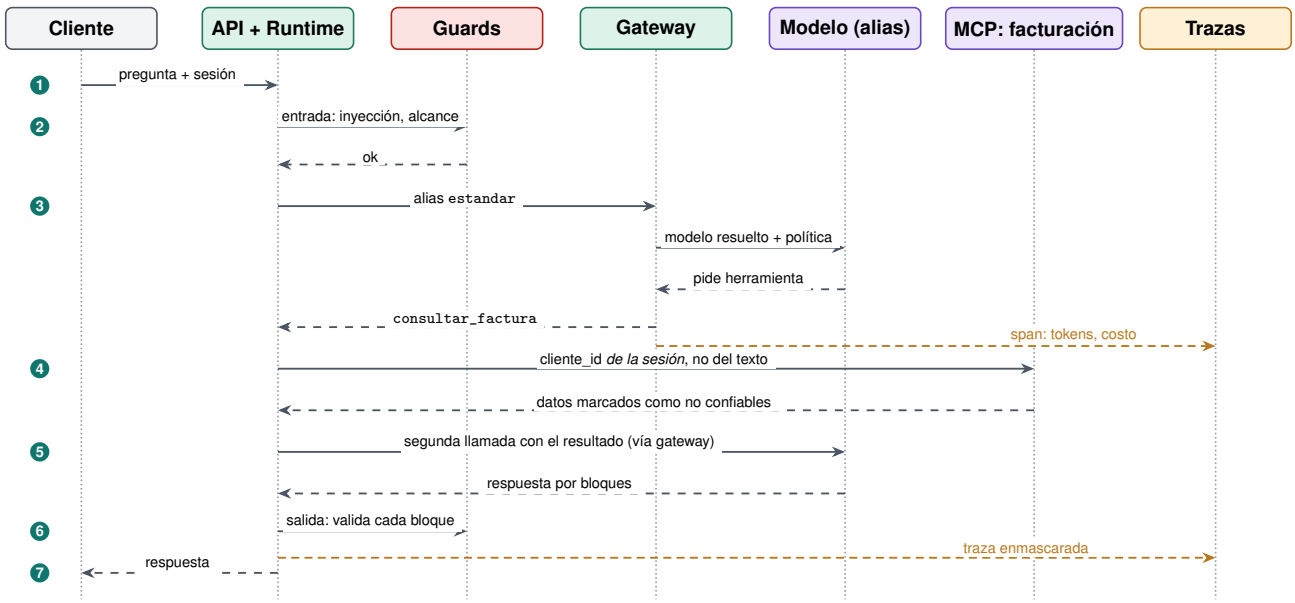


Figura 5: Una consulta de factura de punta a punta. Cada flecha deja un registro en la traza con agente, versión, modelo, tokens, costo y latencia.

La figura 5 sigue el proceso donde la plataforma entra primero: las consultas de factura que hoy atiende un asesor. Dos detalles sostienen la seguridad sin sacrificar la experiencia. El identificador del cliente lo pone el runtime desde la sesión (paso 4), así que el agente no puede consultar a otra empresa aunque se lo pidan. Y el guard de salida valida la respuesta por bloques (paso 6): el cliente la ve aparecer mientras se escribe, pero ningún bloque sale sin haber sido revisado.

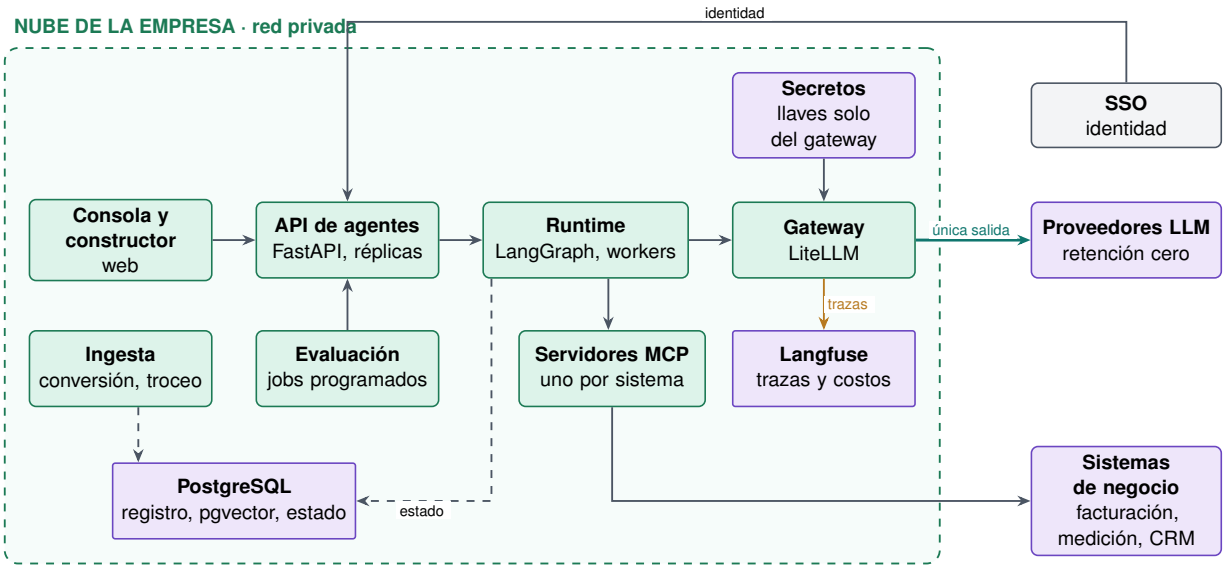


Figura 6: Despliegue en la nube de la empresa. El gateway es la única salida hacia proveedores de modelos; las trazas y los índices quedan dentro de la red privada.

2.4 Despliegue

Decisión: Todo en contenedores dentro de la red de la empresa, con PostgreSQL como almacén único (registro, vectores con pgvector y estado de conversaciones).
Por qué: El volumen de la empresa no justifica una base vectorial dedicada; un solo motor simplifica respaldos, permisos y operación.
Descarté: Base vectorial administrada aparte; plataforma de agentes de un proveedor de nube (ata modelos al proveedor y deja los inventarios fuera de la empresa).

Decisión: Observabilidad autohospedada (Langfuse) con convenciones GenAI de OpenTelemetry [5].
Por qué: Las trazas contienen conversaciones con clientes; aun enmascaradas, conviene que no salgan de la infraestructura de la empresa [6].
Descarté: Servicio SaaS de trazas.

2.5 Seguridad

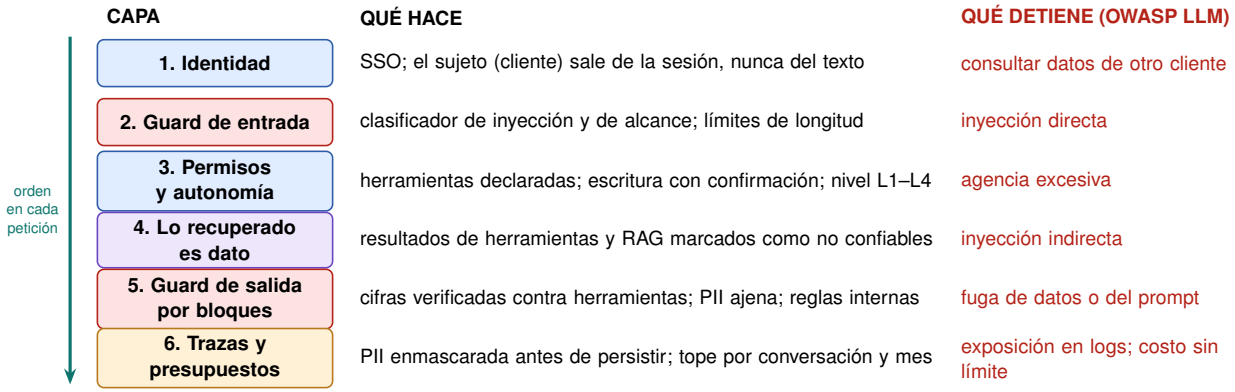


Figura 7: Seis capas que atraviesa cada petición y el riesgo del OWASP Top 10 para LLM [7] que detiene cada una.

Decisión: Los controles son capas de la plataforma que todo agente hereda; ningún creador las puede quitar.
Por qué: Si cada agente implementa su propia seguridad, la seguridad depende de quién lo construyó. La inyección indirecta, por ejemplo, llega dentro de documentos y respuestas de herramientas [8] y solo se contiene tratando todo lo recuperado como dato.
Descarté: Filtros por expresiones regulares por agente: los usé y fallaron en ambos sentidos.

2.6 Clasificación de datos

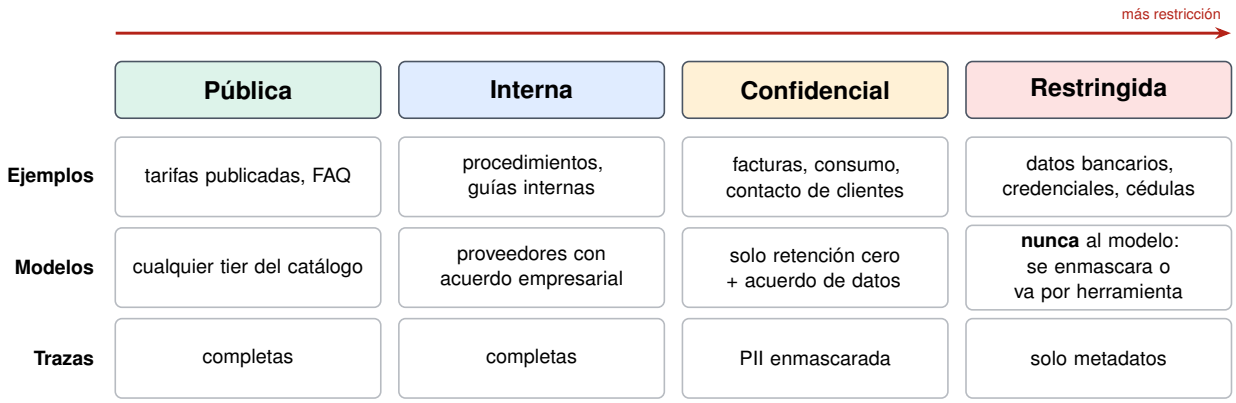


Figura 8: Cuatro niveles de clasificación. El manifiesto declara el nivel del agente y el gateway restringe los modelos y el contenido de las trazas en consecuencia.

Decisión: La clasificación del dato se decide antes que el modelo.
Por qué: Es lo único que no se negocia por calidad o costo: restringe el conjunto de modelos elegibles y lo que se guarda en trazas.
Descarté: Dejar que cada área escoja proveedor según preferencia.

2.7 Inventarios

Los inventarios de agentes, modelos y orquestación no son hojas de cálculo que alguien mantiene: son vistas del plano de control que se llenan solas, porque sin registro no hay camino a producción (tabla 2).

Tabla 2: Inventarios y la pregunta que responde cada uno.

Inventario	Qué registra	Pregunta
Agentes	dueño, área, versión, alias, herramientas, colecciones, clasificación, autonomía, estado, costo	¿Cuántos hay y qué tocan?
Modelos	alias, modelo y versión, proveedor, capacidades verificadas, retención, precio	¿Con qué corren?
Orquestación	herramientas (lectura o escritura, aprobación), colecciones (dueño, vigencia), flujos	¿Qué puede hacer cada uno?

2.8 Modelos

Escojo el modelo por dato y por tarea, en ese orden (figura 9). Y el día que sale un modelo mejor no se cambia en ningún agente: se cambia detrás de un alias, después de pasar por la figura 10.

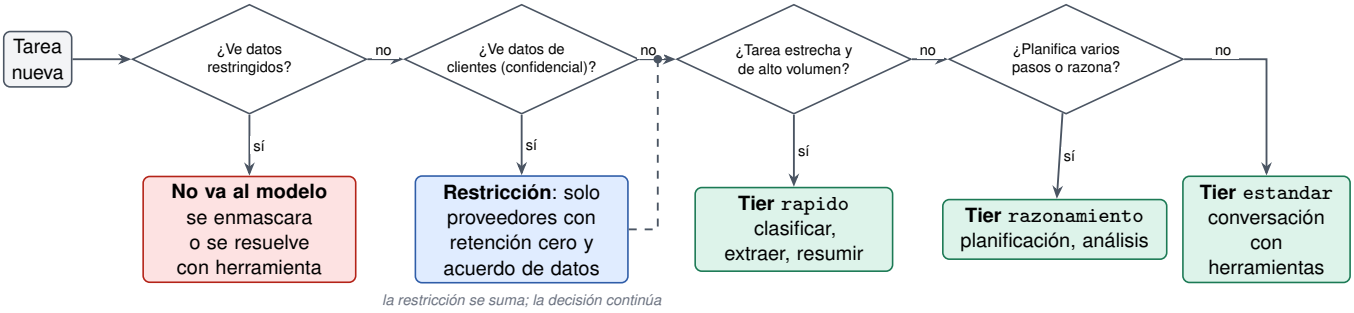


Figura 9: Cómo se asigna un tier: primero el dato, después la tarea.

Decisión: Tres alias (`rapido`, `estandar`, `razonamiento`) en lugar de nombres de modelo; tier fijo por agente al inicio.
Por qué: Las tareas estrechas no necesitan el modelo más caro [9, 10]. El enrutamiento dinámico ahorra más, pero agrega una pieza que evaluar: lo dejo para la fase 3.
Descarté: Un único modelo para todo (caro) o enrutamiento dinámico desde el día uno (complejo).

Decisión: Modelos cerrados por API con retención cero por defecto; sin GPU propias.
Por qué: Hoy dan la mejor capacidad por esfuerzo operativo. Lo local solo se justifica si el dato no puede salir o si el volumen hace más barato operar GPU, y ninguna condición se cumple en la empresa al inicio. Los modelos abiertos entran al catálogo cuando ganan en costo para un tier.
Descarté: Desplegar modelos abiertos propios desde el inicio.

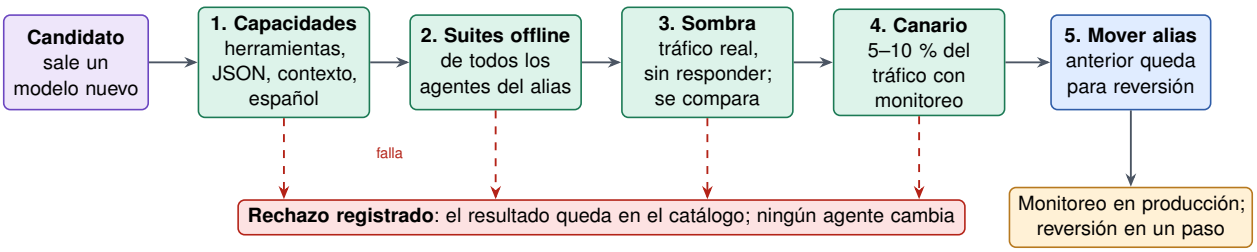


Figura 10: Qué pasa el día que sale un modelo mejor: cada etapa tiene criterio de salida y reversión inmediata.

Decisión: El catálogo registra capacidades *verificadas* (herramientas, JSON válido, contexto efectivo, español) y fija versiones concretas.
Por qué: Un nombre de modelo no garantiza capacidades: lo viví cuando un cambio de modelo rompió la salida estructurada de una función que nadie volvió a probar. Además, un mismo modelo comercial cambia de comportamiento entre versiones [11].
Descarté: Confiar en lo que anuncia el proveedor y usar alias “latest”.

2.9 Conocimiento, RAG y fine-tuning

RAG [12] sirve cuando el conocimiento es extenso, cambia y hay que citarlo. Pero el error más común que veo es usarlo para todo. Antes de llegar a RAG descarto cuatro casos (figura 11): datos de un cliente, cálculos, reglas que siempre aplican y conocimiento pequeño y estable. Para los documentos que sí van a RAG, la ingesta y la consulta siguen la figura 12.

Decisión: Cifras tarifarias en tablas consultables por herramienta; explicaciones y procedimientos en RAG con vigencia obligatoria; troceo distinto por tipo de documento.
Por qué: Una tarifa vencida no es “menos relevante”: es incorrecta, así que se filtra antes de buscar. La búsqueda híbrida con fusión RRF [13] encuentra tanto códigos exactos como preguntas conceptuales, y pocos fragmentos precisos rinden más que muchos [14].
Descarté: RAG para todo, incluidos saldos y tarifas.

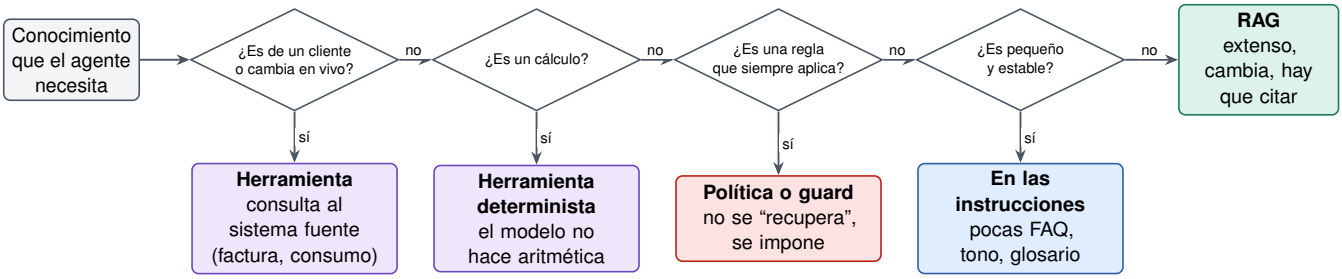


Figura 11: Dónde vive cada tipo de conocimiento: se recorre de izquierda a derecha y se detiene en el primer "sí".

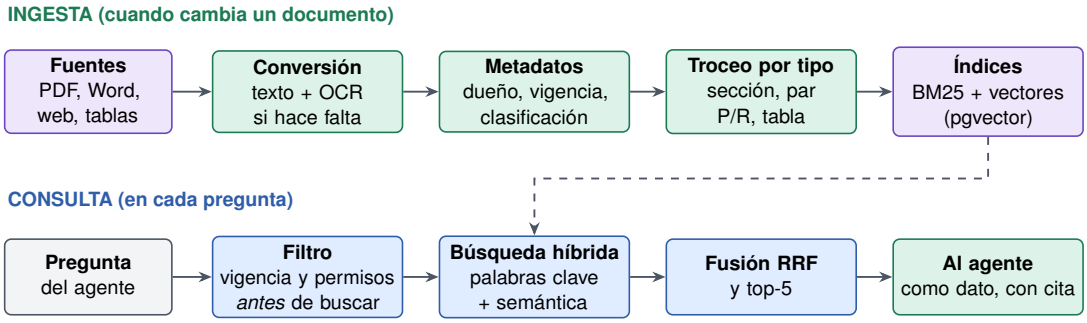


Figura 12: Pipeline de conocimiento para tarifas, procedimientos y preguntas frecuentes.

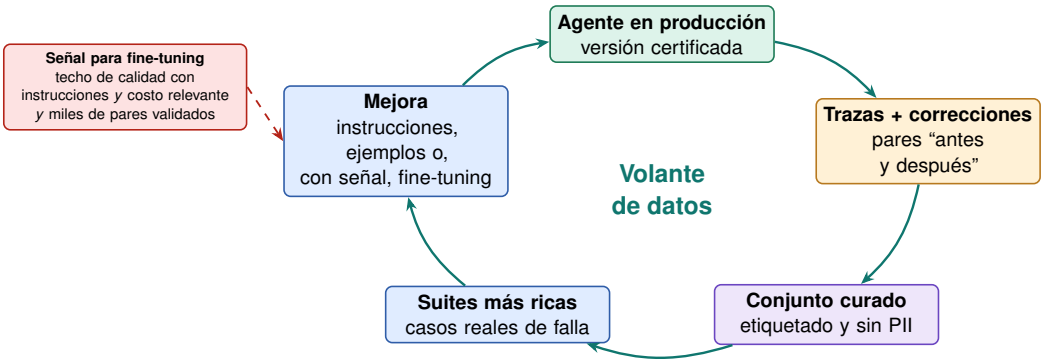


Figura 13: El volante de datos: las correcciones humanas mejoran primero las suites y solo con señal clara alimentan un fine-tuning.

Decisión: No hacer fine-tuning en las fases 0 a 2.
Por qué: No hay datos validados; el conocimiento del negocio cambia cada mes y el fine-tuning inyecta conocimiento peor que la recuperación [15, 16]; y ata la plataforma a un modelo base (choca con P3). Tendría lugar en una tarea estrecha y de alto volumen, como resumir llamadas de ventas, destilando a un modelo pequeño [17, 18] cuando haya techo de calidad, costo relevante y miles de pares validados.
Descarté: Ajustar un modelo propio desde el inicio.

2.10 Orquestación

Decisión: Agente único por defecto; dividir solo por permisos, tiers o evaluación distintos. En procesos regulados el flujo lo decide el código; los traspasos son objetos tipados, nunca el historial completo.
Por qué: En más de 1.600 trazas de siete marcos multiagente, las fallas vinieron sobre todo de especificación, desalineación entre agentes y verificación, no del modelo [19]. El patrón razonar y actuar con herramientas [20] cubre casi todos los casos de la empresa.
Descarté: Arquitectura multiagente desde el inicio con un supervisor LLM decidiendo todo.

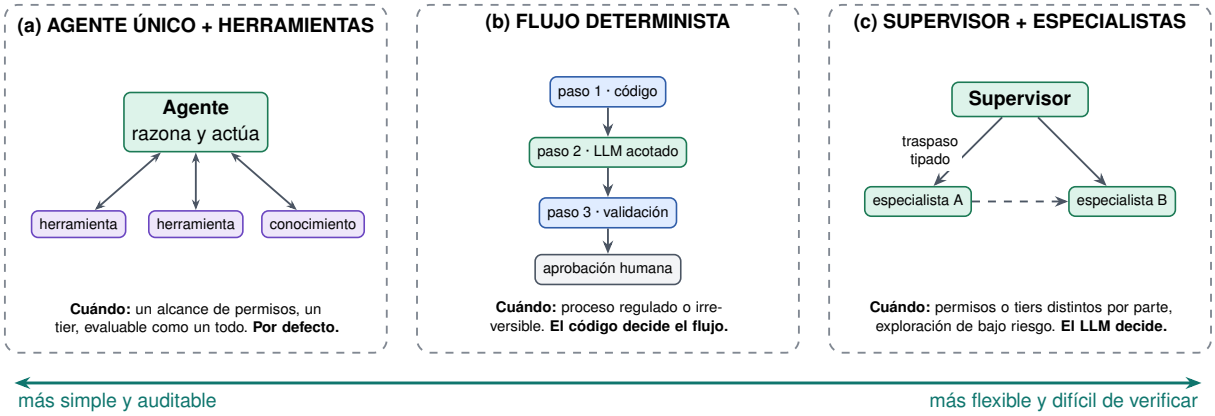


Figura 14: Tres patrones en orden de preferencia.

3 Definición de agente

En resumen

Un **manifiesto declarativo y versionado** que la plataforma sabe ejecutar: para qué existe, de quién es, qué capacidades pide a los catálogos, qué datos ve, cuánta autonomía tiene, cómo se prueba y cuánto puede gastar. No contiene código ni nombres de modelos. Lo que no está en el manifiesto, el agente no lo puede hacer.

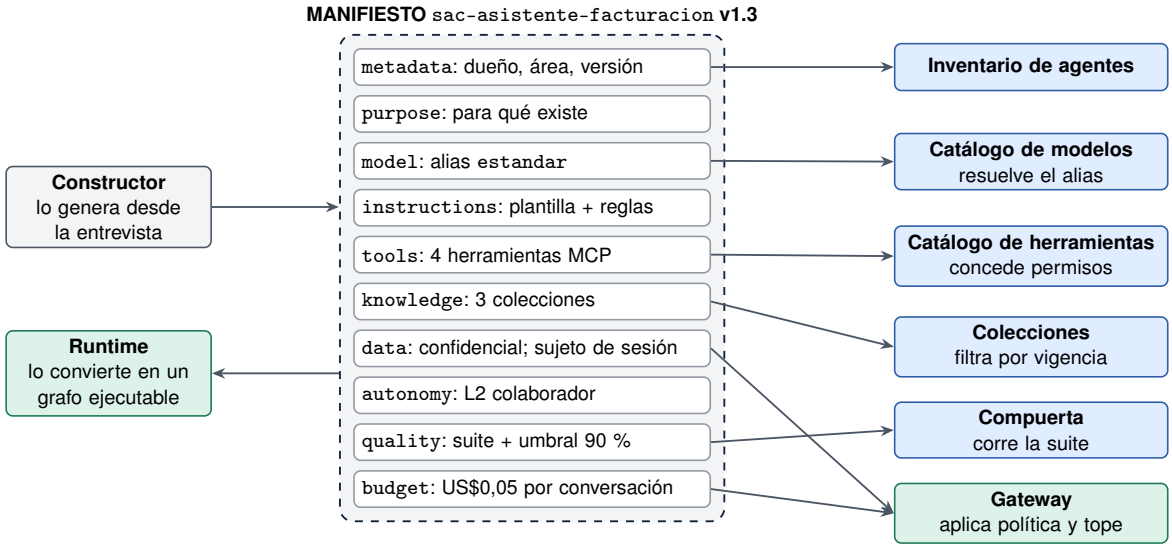


Figura 15: Anatomía del manifiesto. Cada bloque alimenta un componente de la plataforma; el manifiesto completo está en el sección A.

Decisión: El agente es un manifiesto, no código.
Por qué: La plataforma puede razonar sobre un agente sin ejecutarlo (qué datos podría tocar, qué acciones podría disparar) y el inventario se lee directamente del registro. Cumple para los agentes el papel de las *model cards* [21], pero es ejecutable.
Descarté: Agentes como código libre (reproduce las islas) o prompts guardados en una herramienta.

Decisión: La autonomía se declara por agente y se limita a L4 en las fases 1 y 2.
Por qué: La autonomía es una decisión de diseño, no una consecuencia de la capacidad del modelo [22]. Toda acción con efecto externo sobre un cliente pasa por un humano hasta que haya historia de evaluación.
Descarté: Permitir agentes totalmente autónomos desde el inicio.

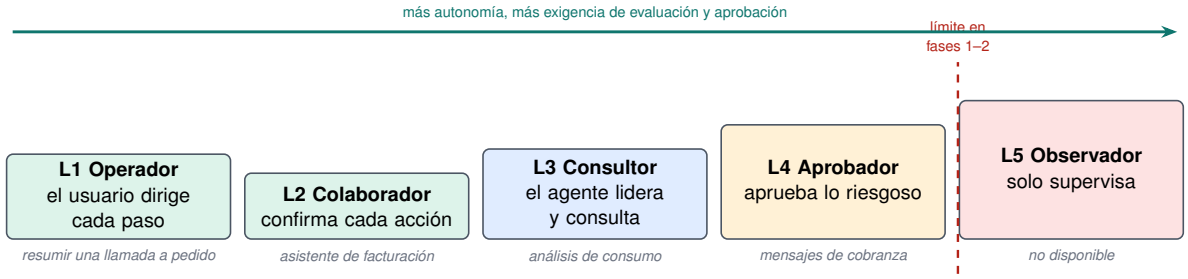


Figura 16: Niveles de autonomía según el papel del usuario [22], con un ejemplo del negocio en cada uno.

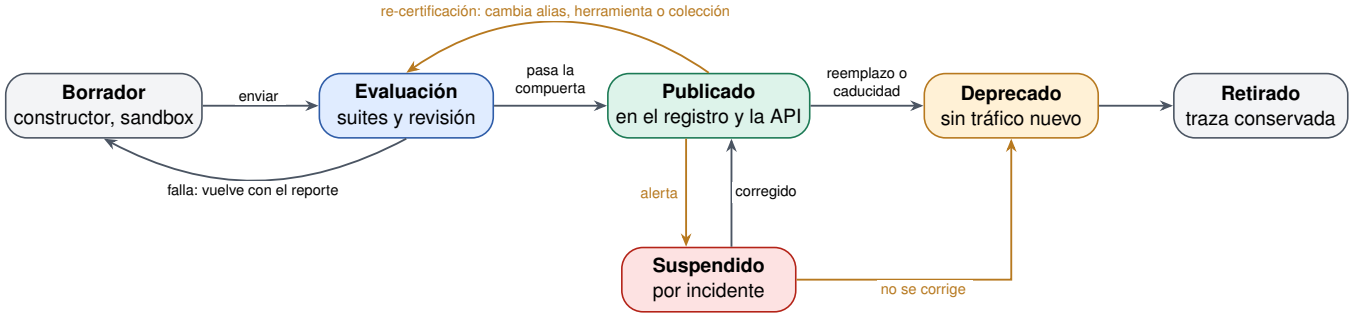


Figura 17: Ciclo de vida. En ámbar, las transiciones que dispara la plataforma.

Decisión: Re-certificación automática y caducidad.
Por qué: Un agente publicado vuelve a evaluación cuando cambia algo de lo que depende (modelo, herramienta, colección), y se depreca si queda sin dueño o sin uso 90 días. Sin esto, toda plataforma de autoservicio se llena de agentes huérfanos.
Descarté: Publicar una vez y revisar solo ante incidentes.

4 Creación de agentes sin escribir prompts

En resumen

No se le pide un prompt. Se le pide lo que sí sabe: la tarea, qué está bien y qué está mal, y ejemplos reales. El *Arquitecto*, un agente de la propia plataforma, lo entrevista; la plataforma compila las instrucciones desde plantillas curadas, hereda los controles y convierte los ejemplos en la primera suite de pruebas.

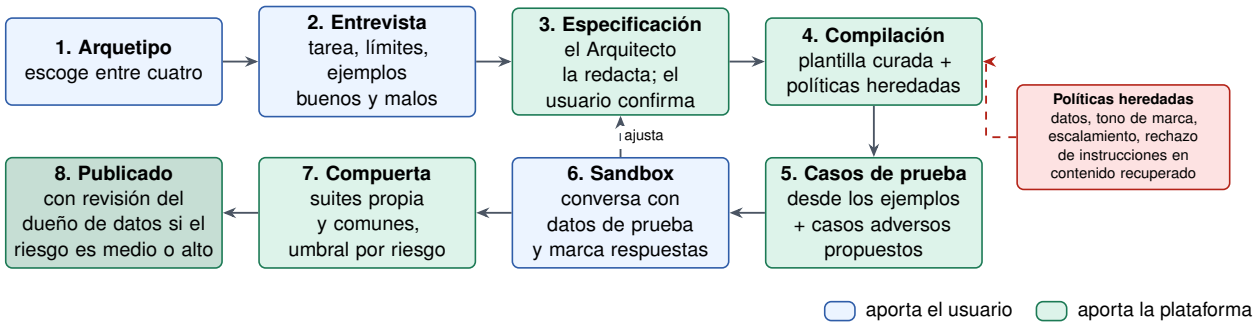


Figura 18: Recorrido del constructor. El usuario aporta el negocio; la plataforma, la técnica.

Decisión: Instrucciones compiladas desde una especificación, no escritas a mano.
Por qué: Cambios de formato que parecen irrelevantes mueven el desempeño de los modelos [23]: escribir prompts es una habilidad técnica frágil. Declarar el *qué* y compilar el *cómo* es la idea de DSPy [24]; aquí la compilación es conservadora (plantillas revisadas) y la optimización automática queda para después. Lo viví: en un sistema que construí, los prompts escritos a mano terminaron ocupando casi toda la configuración y nadie podía revisarlos.
Descarté: Un editor de prompts con buenas prácticas y ejemplos.

Decisión: Cuatro arquetipos y ejemplos que se vuelven pruebas.
Por qué: Pocos arquetipos bien resueltos (tabla 3) cubren la mayoría de las necesidades y cada uno trae plantilla, patrón y suite base. Los ejemplos del usuario, más casos adversos que propone el Arquitecto, dan una suite de 15 a 30 casos que su dueño entiende. El sandbox existe porque los criterios de calidad se descubren al ver respuestas reales [25].
Descarté: Un lienzo libre donde todo es posible.

Tabla 3: Arquetipos iniciales.

Arquetipo	Qué hace	Ejemplo en el negocio	Riesgo
Pregunta sobre documentos	Responde citando colecciones	Procedimientos internos	Bajo
Asistente con datos	Conversa con herramientas de lectura	Facturación	Medio
Redactor	Produce textos; un humano revisa	Cobranza	Medio–alto
Extractor o clasificador	Texto libre a datos estructurados	Llamadas de ventas	Bajo

5 Evaluación y monitoreo

En resumen

Antes: una compuerta con tres capas (suite propia, suite común de seguridad, costo y latencia) y un umbral según el riesgo. Después: muestreo de conversaciones evaluado con los mismos criterios, métricas y alertas por agente, y re-certificación automática ante cualquier cambio. Cada falla en producción se convierte en un caso nuevo de la suite.

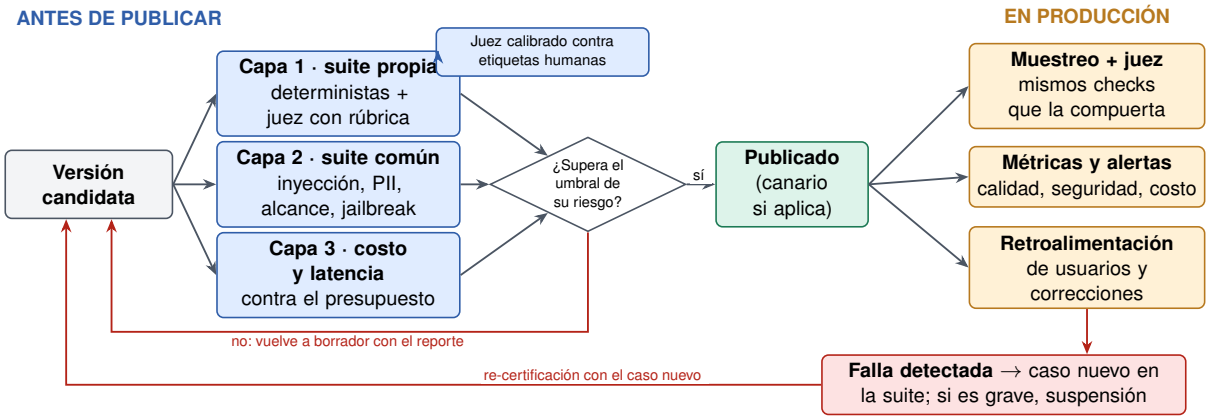


Figura 19: Evaluación en dos tiempos: compuerta antes de publicar y monitoreo en producción.

Decisión: Checks deterministas donde se pueda y juez LLM calibrado donde haga falta criterio.
Por qué: Lo verificable (herramienta correcta, cifra igual a la del sistema, formato) no necesita juicio. Para el resto, el juez escala pero tiene sesgos de posición, verbosidad y familia [26]: uso rúbricas binarias, un juez de otra familia y mido su acuerdo con etiquetas humanas antes de confiar en él.
Descarté: Solo revisión humana (no escala) o solo juez (arrastra sesgos).

Decisión: Costo y latencia son parte de la compuerta.
Por qué: Medir exactitud sin costo lleva a escoger mal [27]; el presupuesto del manifiesto es un criterio de aprobación.
Descarté: Optimizar costo después de publicar.

Tabla 4: Umbrales iniciales de la compuerta por nivel de riesgo (a calibrar).

	Bajo	**Medio**	**Alto**
Definición	Interno, solo lectura	Datos de clientes	Habla con clientes o actúa
Suite propia	≥ 85 %	≥ 90 %	≥ 95 % + revisión humana
Aprobación	Dueño	+ equipo de IA	+ datos y cumplimiento
Despliegue	Directo	Canario	Sombra + canario

En producción se miden por agente cinco dimensiones: calidad (aprobación en la muestra, retroalimentación, escalamientos), seguridad (bloqueos, incidentes de datos, que suspenden el agente de forma automática), costo (por conversación y por área), operación (latencia p95, errores de herramientas) y adopción. Monitorear es necesario porque un agente puede degradarse sin que nadie lo toque: cambian las preguntas, los documentos o el propio modelo [11].

6 Caso aplicado: asistente de facturación

En resumen

Escogí el asistente de facturación de servicio al cliente porque concentra lo difícil en un solo agente: datos confidenciales, conocimiento regulado que cambia cada mes, cifras que no pueden inventarse y una frontera clara entre resolver y escalar. Lo contrasto con cobranzas, donde la ley obliga a que el flujo lo decida el código.

Un cliente empresarial pregunta *¿por qué subió mi factura?* La respuesta exige su consumo medido, su factura, y la explicación de los componentes del costo unitario (generación, transmisión, distribución, comercialización, pérdidas, restricciones). La empresa tiene una ventaja: los datos del medidor inteligente explican la variación con precisión.

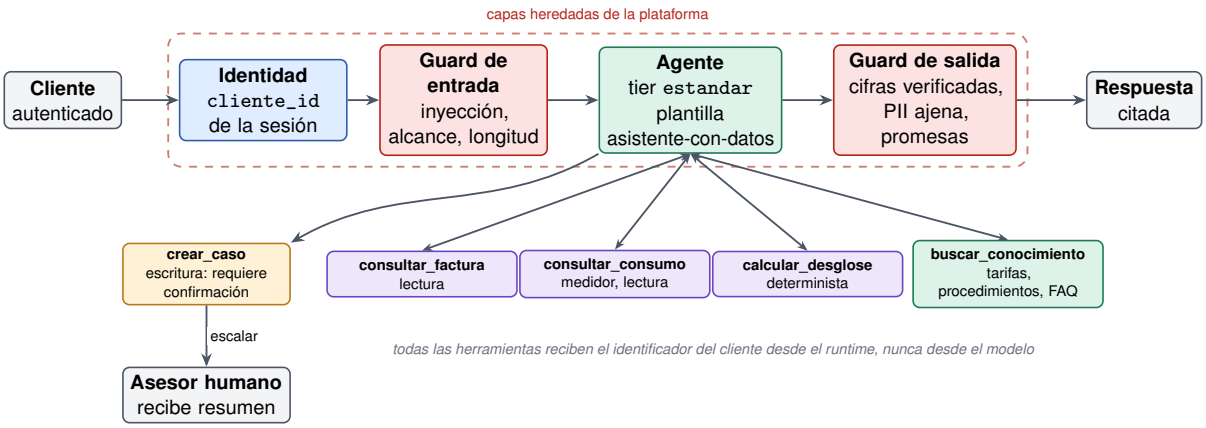


Figura 20: Grafo del asistente de facturación construido desde su manifiesto.

Decisión: El sujeto viene de la sesión y las cifras solo de herramientas.
Por qué: "Dame la factura de la empresa X" no se rechaza con inteligencia: la arquitectura lo hace imposible. Y el guard de salida verifica que toda cifra de la respuesta aparezca en algún resultado de herramienta de esa conversación.
Descarté: Confiar en que las instrucciones del modelo eviten estos errores.

Tabla 5: Cada pregunta va al mecanismo de su tipo de conocimiento (figura 11).

Pregunta	Mecanismo	Por qué
¿Por qué subió mi factura?	Consumo + factura + RAG	Datos propios y explicación regulada
¿Qué es el componente de distribución?	RAG con vigencia	Conocimiento extenso y citable
¿Cuánto debo y cuándo vence?	`consultar_factura`	Dato del cliente
¿Cuánto pagaría con 10 % menos consumo?	`calcular_desglose`	Cálculo determinista
Quiero un acuerdo de pago	`crear_caso` y escalar	Fuera de su autonomía

Éxito de negocio. Conversaciones resueltas sin asesor, tiempo de respuesta y, la más honesta, tasa de reapertura: si el cliente vuelve a preguntar lo mismo por otro canal, el agente trasladó trabajo en vez de ahorrarlo.

Comparación con cobranzas

La Ley 2300 de 2023 [28] solo permite contactar por canales autorizados, prohíbe usar varios canales en la misma semana o contactar más de una vez al día, y limita los horarios a lunes a viernes de 7:00 a.m. a 7:00 p.m. y sábados de 8:00 a.m. a 3:00 p.m.

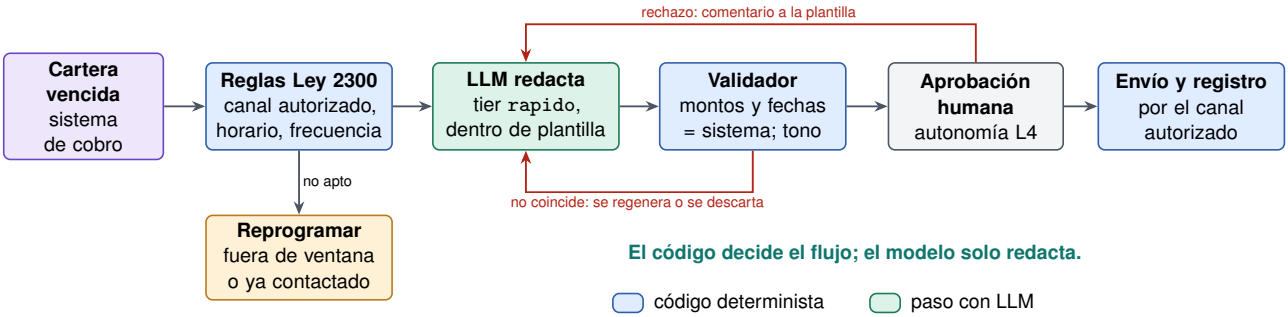


Figura 21: Flujo de cobranza: el modelo ocupa un solo paso acotado y verificado.

Decisión: Cobranza como flujo determinista con aprobación humana (L4).
Por qué: Las reglas legales no pueden depender de que un modelo "se acuerde" de ellas: viven en código y se prueban con casos deterministas. Cada paso con permisos distintos es un componente separado: quien lee la cartera no puede enviar.
Descarté: Un agente autónomo de cobranza con las reglas en su prompt.

7 Hoja de ruta

En resumen

Empezaría por la visibilidad: en dos semanas, un gateway frente a los agentes existentes responde cuántos hay, con qué modelo, qué datos y cuánto cuestan. Luego el núcleo con un agente real (facturación). El autoservicio llega cuando la compuerta existe. Multiagente, enrutamiento dinámico y fine-tuning, solo con señal. La regla: *nunca abrir una puerta antes de tener el control que la vigila*.

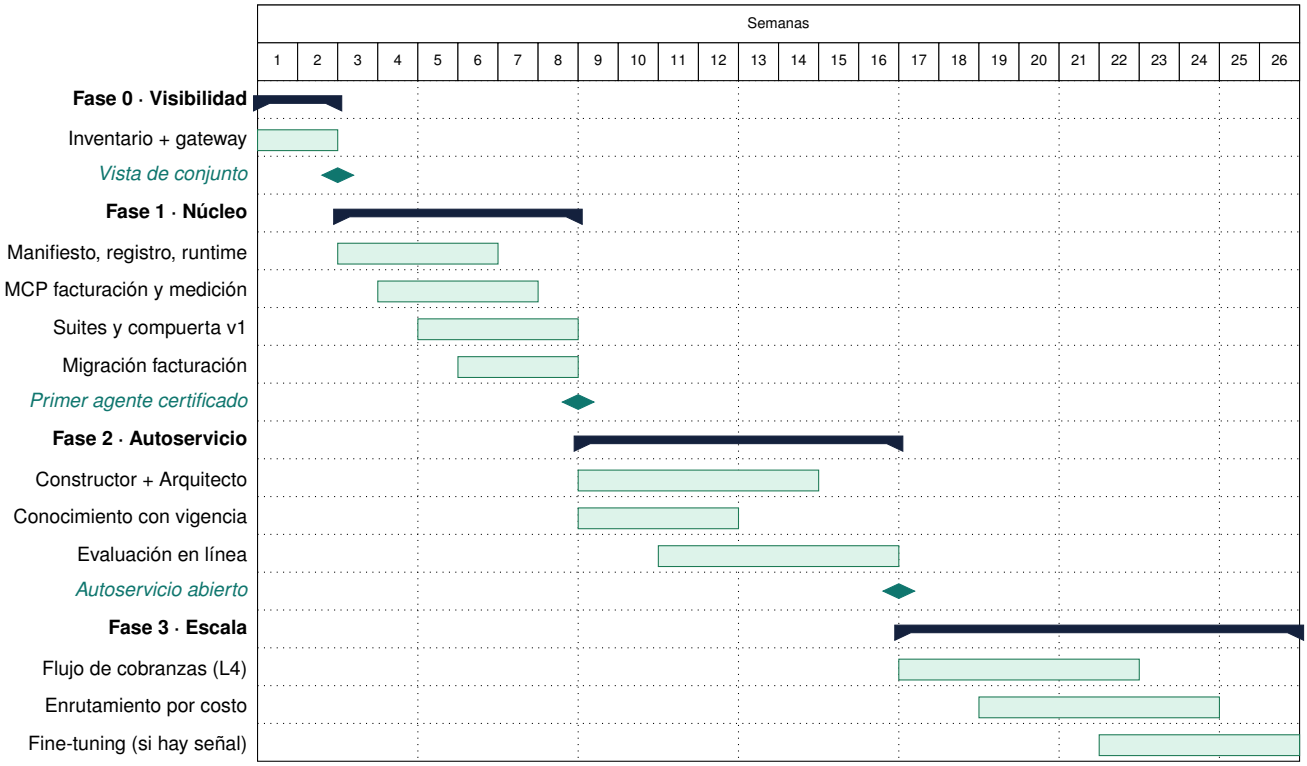


Figura 22: Cronograma grueso en semanas. Los rombos son los hitos que cambian lo que la organización puede hacer.

Tabla 6: Fases: qué entra, qué se adopta y el riesgo que más preocupa.

Fase	Qué entra	Se adopta	Riesgo y mitigación
0 · Visibilidad sem. 1–2	Inventario de agentes actuales; gateway con llaves centralizadas; costo por área	LiteLLM, Langfuse	Resistencia de áreas: solo cambian URL y llave
1 · Núcleo sem. 3–8	Manifiesto, registro, runtime con guards, MCP de facturación y medición, compuerta v1, migrar facturación	LangGraph, pgvector	Sobrediseñar: se diseña contra un agente real
2 · Autoservicio sem. 9–16	Constructor y Arquitecto, ingesta con vigencia, evaluación en línea, caducidad	SSO existente	Agentes mediocres: compuerta, dueño y caducidad
3 · Escala sem. 17–26	Cobranzas L4, enrutamiento por costo, cobro por área, fine-tuning si hay señal	—	Complejidad multiagente: traspasos tipados

Decisión: Construir solo lo específico del negocio; adoptar el resto.
Por qué: Se construyen manifiesto, registro, constructor, compuerta y conectores MCP: es el conocimiento que ningún proveedor tiene. Se adoptan gateway, runtime, vectores, observabilidad y SSO.
Descarté: Comprar una plataforma de agentes completa: más rápida en la fase 2, pero ata modelos y deja los inventarios fuera de la empresa.

El riesgo que más me preocupa es de confianza: que un agente exponga datos de un cliente a otro o ejecute una acción no autorizada. Por eso sus tres defensas (sujeto desde la sesión, escritura con confirmación, guard de salida por bloques) están en la fase 1, antes del primer agente publicado. Es coherente con el NIST AI RMF [29], la ISO/IEC 42001 [30] y el eje de mitigación de riesgos del CONPES 4144 [31].

8 Próximos pasos

Una plataforma de agentes se gobierna por su forma, no por sus reglas. Si un agente es un manifiesto que pide capacidades a catálogos curados, y si el único camino a producción pasa por registro, compuerta y gateway, las preguntas que hoy la empresa no puede responder se responden solas. Y las que vendrán (qué pasa si cambio de modelo, quién aprobó esto, por qué respondió aquello) tienen una traza donde buscarlas.

Junto a este documento entrego un prototipo que implementa un corte vertical de la arquitectura: manifiesto y registro, catálogo con alias, gateway con costo por agente, runtime con guards heredados, herramientas MCP sobre datos sintéticos, la compuerta de evaluación y el *Arquitecto*, el agente que guía la creación de agentes nuevos.

Lo que haría la primera semana es validar cinco supuestos de los que depende el plan: el inventario real de agentes y proveedores; si facturación, medición y CRM exponen API; el proveedor de identidad; los acuerdos de datos vigentes con proveedores de modelos; y quién es dueño de cada colección de conocimiento.

Referencias

[1] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS)*, 2015.

[2] Gartner. Gartner predicts over 40 % of agentic AI projects will be canceled by end of 2027. Comunicado de prensa, June 2025.

[3] Anthropic. Building effective agents. Anthropic Engineering, 2024. URL `https://www.anthropic.com/engineering/building-effective-agents`.

[4] Model Context Protocol. Model context protocol specification, 2025. URL `https://modelcontextprotocol.io/specification`.

[5] OpenTelemetry. Semantic conventions for generative AI systems, 2025. URL `https://opentelemetry.io/docs/specs/semconv/gen-ai/`.

[6] Congreso de la República de Colombia. Ley estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales, 2012.

[7] OWASP Foundation. OWASP top 10 for large language model applications 2025, 2024. URL `https://genai.owasp.org/llm-top-10/`.

[8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications

with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[9] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[10] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations (ICLR)*, 2025.

[11] Lingjiao Chen, Matei Zaharia, and James Zou. How is ChatGPT's behavior changing over time? *Harvard Data Science Review*, 6(2), 2024.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[13] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, 2009.

[14] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

[15] Oded Ovadia, Menachem Brief, Moshik Mishaeli, and Oren Elisha. Fine-tuning or retrieval? comparing knowledge injection in LLMs. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[16] Zorik Gekhman, Gal Yona, Roee Aharoni, Matan Eyal, Amir Feder, Roi Reichart, and Jonathan Herzig. Does fine-tuning LLMs on new knowledge encourage hallucinations? In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[18] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics (ACL)*, 2023.

[19] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS), Datasets and Benchmarks Track*, 2025.

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[21] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019.

[22] K. J. Kevin Feng, David W. McDonald, and Amy X. Zhang. Levels of autonomy for AI agents. *arXiv preprint arXiv:2506.12469*, 2025.

[23] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024.

[24] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

[25] Shreya Shankar, J. D. Zamfirescu-Pereira, Björn Hartmann, Aditya G. Parameswaran, and Ian Arawjo. Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2024.

[26] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023.

[27] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

[28] Congreso de la República de Colombia. Ley 2300 de 2023, por medio de la cual se establecen medidas que protejan el derecho a la intimidad de los consumidores, 2023.

[29] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, NIST, 2023.

[30] ISO/IEC. ISO/IEC 42001:2023 information technology — artificial intelligence — management system, 2023.

[31] Departamento Nacional de Planeación. Documento CONPES 4144: Política nacional de inteligencia artificial. Consejo Nacional de Política Económica y Social, Bogotá, 14 de febrero de 2025, 2025.

A Manifiesto completo del asistente de facturación

```
apiVersion: charter/v1
kind: Agent
metadata:
  id: sac-asistente-facturacion
  version: 1.3.0
  owner: lider.sac@energia.example          # sin dueño no se publica
  area: servicio-al-cliente
  archetype: asistente-con-datos
spec:
  purpose: >
    Responder dudas de clientes empresariales sobre su factura de
    energía y explicar variaciones de consumo con datos del medidor.
  model: { alias: estandar, fallback: rapido }   # el catálogo resuelve el modelo
  instructions:
    template: asistente-con-datos@4               # plantilla curada, no prompt libre
    tone: cercano-profesional
    must:     [citar la fuente de cada cifra, ofrecer escalar a un asesor]
    must_not: [prometer descuentos o acuerdos de pago, hablar de otro cliente]
  tools:
```

```yaml
    - mcp: facturacion/consultar_factura          # solo lectura
    - mcp: medicion/consultar_consumo             # solo lectura
    - mcp: tarifas/calcular_desglose              # determinista
    - mcp: soporte/crear_caso
      requires_confirmation: true                 # escritura: confirma el usuario
  knowledge: [tarifas-metodologia, procedimientos-sac, faq-facturacion]
  data:
    classification: confidencial
    subject_from: session.cliente_id              # nunca del texto del usuario
    mask_pii_in_traces: true
  autonomy: L2                                    # colaborador: confirma escrituras
  quality:
    suite: evals/sac-facturacion.yaml
    min_pass_rate: 0.90
    inherit: [red-team-base, pii-base]
  budget: { max_usd_per_conversation: 0.05, monthly_cap_usd: 300 }
```